

1. Trees	2
1.1 Binary Search Tree (BST)	2
1.2 RED-BLACK Trees	5
1.3 AVL Trees	12
2. Hash Tables	16
2.1 Բախումների լուծում - շղթայում (Separate Chaining)	18
2.2 Բաց հասցեավորում - Open Addressing	19
3. Dynamic Programming	20
3.1 Ֆիբոնաչիի թվեր (Fibonacci)	20
3.2 Rod Cutting Problem	22
3.3 Coin Change Problem	24
3.4 Knapsack Problem	26
4. Greedy Algorithms	28
4.1 Activity Selection	28
4.2 Huffman Coding	29
4.3 Minimum Spanning Tree	30

1. Trees

1.1 Binary Search Tree (BST)

BST-ն կառուցված է հանգույցներից, որոնցից յուրաքանչյուրն ունի առավելագույնը երկու զավակ՝ ձախ և աջ: **BST հատկություն**՝ ձախ ենթաճառի բոլոր բանալիները փոքր են հանգույցի բանալուց, իսկ աջ ենթաճառինը՝ մեծ կամ հավասար: Այս հատկության շնորհիվ որոնումը, ավելացումը և ջնջումը կատարվում են $O(h)$ ժամանակում, որտեղ h -ը ճառի բարձրությունն է: Վատագույն դեպքում՝ $O(n)$, լավագույն դեպքում՝ $O(\log(n))$:

Հիմնական գործողություններ՝

- Որոնում (Search) – սկսում ենք արմատից, համեմատում բանալին և շարունակում ձախ կամ աջ՝ կախված համեմատությունից:
- Տեղադրում (Insert) – գտնում ենք այն դիրքը, որտեղ պետք է լինի նոր հանգույցը (որպես տերև) և կցում ենք այն:
- Ջնջում (Delete) – երեք դեպք՝ հանգույցը չունի զավակ, ունի մեկ զավակ, ունի երկու զավակ (ապա գտնում ենք հաջորդողը/նախորդողը և տեղափոխում):

Շրջանցումներ (Traversals)

- Inorder (left, root, right) - տալիս է դասավորված հաջորդականություն
- Preorder (root, left, right)
- Postorder (left, right, root)

```
class BSTNode:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.parent = None

def bst_search(root, key):
    while root and root.key != key:
        if key < root.key:
            root = root.left
        else:
            root = root.right
```

```

        return root

def bst_insert(root, z):
    y = None
    x = root
    while x:
        y = x
        if z.key < x.key:
            x = x.left
        else:
            x = x.right
    z.parent = y
    if y is None:
        return z          # new root
    elif z.key < y.key:
        y.left = z
    else:
        y.right = z
    return root

def transplant(root, u, v):
    if u.parent is None:
        root = v
    elif u == u.parent.left:
        u.parent.left = v
    else:
        u.parent.right = v
    if v:
        v.parent = u.parent
    return root

def bst_delete(root, z):
    if z.left is None:
        root = transplant(root, z, z.right)
    elif z.right is None:
        root = transplant(root, z, z.left)
    else:
        y = bst_minimum(z.right)
        if y.parent != z:
            root = transplant(root, y, y.right)

```

```

        y.right = z.right
        y.right.parent = y
        root = transplant(root, z, y)
        y.left = z.left
        y.left.parent = y
    return root

def bst_minimum(root:BSTNode):
    current = root
    while current.left is not None:
        current = current.left
    return current

def bst_maximum(root:BSTNode):
    current = root
    while current.right is not None:
        current = current.right
    return current

def inorder(root:BSTNode):
    if root is not None:
        inorder(root.left)
        print(root.key)
        inorder(root.right)

def preorder(root:BSTNode):
    if root is not None:
        print(root.key)
        preorder(root.left)
        preorder(root.right)

def postorder(root:BSTNode):
    if root is not None:
        postorder(root.left)
        postorder(root.right)
        print(root.key)

```

1.2 RED-BLACK Trees

RBT-ն self-balanced BST է, որտեղ յուրաքանչյուր հանգույց ներկված է կարմիր կամ սև՝ հետևյալ հատկություններով՝

1. Արմատը սև է
2. Բոլոր տերմները սև են
3. Կարմիր հանգույցի երկու զավակներն էլ սև են (կարմիրը չի կարող կարմիր զավակ ունենալ):
4. Ցանկացած հանգույցից մինչև տերև բոլոր ուղիներում սև հանգույցների քանակը նույնն է (սև բարձրություն):

Այս հատկությունները երաշխավորում են, որ ծառի բարձրությունը $O(\log n)$ է, ուստի գործողությունները կատարվում են լոգարիթմական ժամանակում:

```
class Node:
    """
    Node class for Red-Black-Tree
    """
    def __init__(self, key):
        self.val = key
        self.parent = None
        self.left = None
        self.right = None
        self.color = False # False for RED, True for Black
        self.height = 0
```

```
class Red_Black_Tree:
    """
    Rules.
    1. Every node is either RED or BLACK
    2. Root is always BLACK
    3. Each leaf (None or NIL) is BLACK
    4. If the node is RED then its children must be BLACK
    5. Any path from node to leaf must contain the same amount of
    BLACK nodes (Black-Height)
    """
    def __init__(self):
        self.root = None

    def __get_height(self, node:Node) -> int:
        return node.height if node else -1
```

```

def __update_height(self, node:Node) -> None:
    if node:
        node.height = 1 + max(self.__get_height(node.left),
self.__get_height(node.right))

def left_rotate(self, z:Node) -> None:
    y = z.right
    z.right = y.left
    if y.left is not None:
        y.left.parent = z
    y.parent = z.parent
    if z.parent is None:
        self.root = y
    elif z == z.parent.left:
        z.parent.left = y
    else:
        z.parent.right = y
    y.left = z
    z.parent = y

    self.__update_height(y)
    self.__update_height(z)

def right_rotate(self, z:Node) -> None:
    y = z.left
    z.left = y.right
    if y.right is not None:
        y.right.parent = z

    y.parent = z.parent
    if z.parent is None:
        self.root = y
    elif z == z.parent.right:
        z.parent.right = y
    else:
        z.parent.left = y

    y.right = z
    z.parent = y

```

```

self.__update_height(y)
self.__update_height(z)

def __insert_fixup(self, z:Node) -> None:
    while z.parent and z.parent.parent and z.parent.color == False:
        if z.parent == z.parent.parent.left:
            y = z.parent.parent.right # # uncle

            # case1: uncle is "RED"
            if y and y.color == False:
                z.parent.color = True
                y.color = True
                z.parent.parent.color = False
                z = z.parent.parent
            else:
                # case2: uncle is "BLACK" and z is the right-child
                if z == z.parent.right:
                    z = z.parent
                    self.left_rotate(z)

                # case3: uncle is "BLACK" and z is the left-child
                z.parent.color = True
                z.parent.parent.color = False
                self.right_rotate(z.parent.parent)
        else:
            y = z.parent.parent.left # uncle

            # case4: uncle is "RED"
            if y and y.color == False:
                z.parent.color = True
                y.color = True
                z.parent.parent.color = False
                z = z.parent.parent
            else:
                # case5: uncle is "BLACK" and z is the left-child
                if z == z.parent.left:
                    z = z.parent
                    self.right_rotate(z)

```

```

        # case6: uncle is "BLACK" and z is the right-child
        z.parent.color = True
        z.parent.parent.color = False
        self.left_rotate(z.parent.parent)
    self.root.color = True

def insert(self, key) -> None:
    x = self.root
    y = None
    z = Node(key)
    while x is not None:
        y = x
        if z.val < x.val:
            x = x.left
        else:
            x = x.right
    z.parent = y
    if y is None:
        self.root = z
    elif z.val < y.val:
        y.left = z
    else:
        y.right = z
    z.left = None
    z.right = None
    z.color = False

    temp = z
    while temp is not None:
        self.__update_height(temp)
        temp = temp.parent

    self.__insert_fixup(z)
def __transplant(self, u:Node, v:Node) -> None:
    if u.parent is None:
        self.root = v
    elif u == u.parent.left:
        u.parent.left = v
    else:
        u.parent.right = v

```



```

        if v is not None:
            v.parent = u.parent

def delete(self, z: Node) -> None:
    y = z
    y_original_color = y.color
    if z.left is None:
        x = z.right
        self.__transplant(z, z.right)
        parent_to_update = z.parent
    elif z.right is None:
        x = z.left
        self.__transplant(z, z.left)
        parent_to_update = z.parent
    else:
        y = self.minimum(z.right)
        y_original_color = y.color
        x = y.right
        if y.parent == z:
            if x is not None:
                x.parent = y
                parent_to_update = y
            else:
                parent_to_update = y.parent
                self.__transplant(y, y.right)
                y.right = z.right
                y.right.parent = y
        else:
            self.__transplant(z, y)
            y.left = z.left
            y.left.parent = y
            y.color = z.color

    temp = parent_to_update
    while temp is not None:
        self.__update_height(temp)
        temp = temp.parent

    if y_original_color == True:
        self.__delete_fixup(x, parent_to_update)

```

```

def __delete_fixup(self, x: Node, parent: Node) -> None:
    while x != self.root and (x is None or x.color == True):

        if x == parent.left or (x is None and parent.left is None):
            w = parent.right
            if w and w.color == False:
                w.color = True
                parent.color = False
                self.left_rotate(parent)
                w = parent.right

            if w and (not w.left or w.left.color == True) and (not
w.right or w.right.color == True):
                w.color = False
                x = parent
                parent = x.parent if x else None
            else:
                if w and (not w.right or w.right.color == True):
                    if w.left: w.left.color = True
                    w.color = False
                    self.right_rotate(w)
                    w = parent.right
                if w:
                    w.color = parent.color
                    parent.color = True
                    if w.right: w.right.color = True
                    self.left_rotate(parent)
                x = self.root
        else: # Mirror Case
            w = parent.left
            if w and w.color == False:
                w.color = True
                parent.color = False
                self.right_rotate(parent)
                w = parent.left

            if w and (not w.left or w.left.color == True) and (not
w.right or w.right.color == True):
                w.color = False
                x = parent

```

```

        parent = x.parent if x else None
    else:
        if w and (not w.left or w.left.color == True):
            if w.right: w.right.color = True
            w.color = False
            self.left_rotate(w)
            w = parent.left
        if w:
            w.color = parent.color
            parent.color = True
            if w.left: w.left.color = True
            self.right_rotate(parent)
        x = self.root

    if x:
        x.color = True

def search(self, k) -> Node:
    current = self.root
    while current is not None and k != current.val:
        if k < current.val:
            current = current.left
        else:
            current = current.right
    return current

def search_with_color(self, k) -> dict:
    current = self.root
    while current and k != current.val:
        if k < current.val:
            current = current.left
        else:
            current = current.right
    return {
        "Node" : current,
        "Color" : "BLACK" if current.color else "RED"
    }

def height(self, x:Node = "not_given") -> int:
    if x == "not_given":
        x = self.root

```

```

        if x is None:
            return -1
        return x.height

def black_height(self, x:Node = None) -> int:
    if x is None:
        x = self.root
    h = 1 # None (Nil) is included
    current = x
    while current is not None:
        if current.color == True:
            h += 1
        current = current.left
    return h

```

1.3 AVL Trees

AVL ծառը հավասարակշռված BST է, որտեղ յուրաքանչյուր հանգույցի համար ձախ և աջ ենթածառերի բարձրությունների տարբերությունը (բալանսի գործոն) ≤ 1 : Այս պայմանն ապահովում է $O(\log N)$ բարձրություն: Տեղադրելուց կամ ջնջելուց հետո հաշվարկում ենք բարձրությունները և կատարում պտույտներ (LL, RR, LR, RL)՝ հավասարակշռությունը վերականգնելու համար:

```

class AVLNode:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1 # նոր հանգույցի բարձրությունը 1 է (սերև)

class AVLTree:
    # օժանդակ ֆունկցիաներ
    def _height(self, node):
        return node.height if node else 0

    def _balance(self, node):
        return self._height(node.left) - self._height(node.right) if node
else 0

```

```

def _update_height(self, node):
    if node:
        node.height = 1 + max(self._height(node.left),
self._height(node.right))

# պսնւյսնէր
def _right_rotate(self, y):
    x = y.left
    T2 = x.right
    # պսնւյս
    x.right = y
    y.left = T2
    # Թարմացնել բարձրությունները
    self._update_height(y)
    self._update_height(x)
    return x    # նոր արմատ

def _left_rotate(self, x):
    y = x.right
    T2 = y.left
    y.left = x
    x.right = T2
    self._update_height(x)
    self._update_height(y)
    return y

# սեղադրում
def insert(self, root, key):
    # 1. Սովորական BST սեղադրում
    if not root:
        return AVLNode(key)
    if key < root.key:
        root.left = self.insert(root.left, key)
    elif key > root.key:
        root.right = self.insert(root.right, key)
    else:
        return root    # կրկնվող բանալիներ չենք ավելացնում

    # 2. Թարմացնել բարձրությունը
    self._update_height(root)

```

```

# 3. Մտանալ հավասարակշռման գործոնը
balance = self._balance(root)

# 4. Եթե հանգույցը անհավասարակշռված է, կիրառել համապատասխան
այսույժը

# Left Left
if balance > 1 and key < root.left.key:
    return self._right_rotate(root)
# Right Right
if balance < -1 and key > root.right.key:
    return self._left_rotate(root)
# Left Right
if balance > 1 and key > root.left.key:
    root.left = self._left_rotate(root.left)
    return self._right_rotate(root)
# Right Left
if balance < -1 and key < root.right.key:
    root.right = self._right_rotate(root.right)
    return self._left_rotate(root)

return root

# գտնել նվազագույն բանալիով հանգույցը (աջակցում է delete-ին)
def _min_value_node(self, node):
    current = node
    while current.left:
        current = current.left
    return current

# ջնջում
def delete(self, root, key):
    # 1. Սովորական BST ջնջում
    if not root:
        return root
    if key < root.key:
        root.left = self.delete(root.left, key)
    elif key > root.key:
        root.right = self.delete(root.right, key)

```

```

else:
    # զսանձ ջնջվող հանգույցը
    if root.left is None:
        return root.right
    elif root.right is None:
        return root.left
    # երկու գավակներն էլ կան
    temp = self._min_value_node(root.right)
    root.key = temp.key
    root.right = self.delete(root.right, temp.key)

if root is None:
    return root

# 2. Թարմացնել բարձրությունը
self._update_height(root)

# 3. Մտնալ հավասարակշռման գործոնը
balance = self._balance(root)

# 4. Վերականգնել հավասարակշռությունը

# Left Left
if balance > 1 and self._balance(root.left) >= 0:
    return self._right_rotate(root)
# Left Right
if balance > 1 and self._balance(root.left) < 0:
    root.left = self._left_rotate(root.left)
    return self._right_rotate(root)
# Right Right
if balance < -1 and self._balance(root.right) <= 0:
    return self._left_rotate(root)
# Right Left
if balance < -1 and self._balance(root.right) > 0:
    root.right = self._right_rotate(root.right)
    return self._left_rotate(root)

return root

# inorder traversal (սպուլն է դասավորված հաջորդականությամբ)

```

```
def inorder(self, root):
    if root:
        self.inorder(root.left)
        print(root.key, end=' ')
        self.inorder(root.right)
```

2. Hash Tables

Hash աղյուսակը տվյալների կառուցվածք է, որն օգտագործում է **հեշ ֆունկցիա**, բանալին քարտեզագրելու համար աղյուսակի ինդեքսի (slot): Գաղափարն է՝ $O(1)$ միջին ժամանակում կատարել որոնում, տեղադրում և ջնջում:

```
class Node:
    def __init__(self, key, value):
        self.key = key
        self.value = value
        self.next = None

class Hash_Table:
    """
    The default capacity is 11 elements.
    """
    def __init__(self, capacity:int = 11):
        self.capacity = capacity
        self.size = 0
        self.table = [None] * capacity

    def _hash(self, key) -> int:
        hash_value = 0
        prime_multiplier = 31

        for char in str(key):
            # # ord(char) -> returns the ASCII number of 'char'
            hash_value = (hash_value * prime_multiplier) + ord(char)

        return hash_value % self.capacity

    def _resize(self) -> None:
```



```

old_table = self.table
self.capacity = self.capacity * 2 + 1
self.table = [None] * self.capacity
self.size = 0

for node in old_table:
    current = node
    while current:
        self.insert(current.key, current.value)
        current = current.next

def insert(self, key, value) -> None:
    if self.size / self.capacity > 0.7:
        self._resize()

    index = self._hash(key)

    if self.table[index] is None:
        self.table[index] = Node(key, value)
        self.size += 1
    else:
        current = self.table[index]
        while current:
            if current.key == key:
                current.value = value
                return
            current = current.next
        new_node = Node(key, value)
        new_node.next = self.table[index]
        self.table[index] = new_node
        self.size += 1

def search(self, key):
    index = self._hash(key)
    current = self.table[index]
    while current:
        if current.key == key:
            return current.value
        current = current.next
    raise KeyError(key)

```

```

def remove(self, key):
    index = self._hash(key)
    previous = None
    current = self.table[index]

    while current:
        if current.key == key:
            if previous:
                previous.next = current.next
            else:
                self.table[index] = current.next
            self.size -= 1
            return
        previous = current
        current = current.next
    raise KeyError(key)

```

2.1 Բախումների լուծում - շղթայում (Separate Chaining)

Յուրաքանչյուր **slot** պարունակում է linked list (կամ այլ կոնտեյներ): Բոլոր բանալիները, որոնց հեշ արժեքը նույնն է, պահվում են այդ ցուցակում: Բեռնվածության գործակիցը՝ $\alpha = n/m$ (միջինում ցուցակի երկարությունը): Անհաջող որոնման ակնկալվող ժամանակը $\Theta(1+\alpha)$ է, իսկ հաջողինը՝ $\Theta(1+\alpha)$:

```

class ChainedHash:
    def __init__(self, size):
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        return key % self.size

    def insert(self, key):
        self.table[self._hash(key)].append(key)

    def search(self, key):
        return key in self.table[self._hash(key)]

    def delete(self, key):
        self.table[self._hash(key)].remove(key)

```

2.2 Բաց հասցեավորում - Open Addressing

Բոլոր տարրերը պահվում են անմիջապես ադյուսակում: Բախման դեպքում կատարվում է զոնդավորում (probe sequence)՝ մինչև դատարկ տեղ գտնելը:

Linear Probing - $h(k, i) = (h'(k) + i) \bmod m$: Առավելությունն է՝ քեշի լավ օգտագործումը, թերությունը՝ առաջնային կլաստերացումը (երկար շղթաներ):

```
class LinearProbing:
    def __init__(self, size):
        self.size = size
        self.table = [None]*size
    def _hash(self, key, i):
        return (key + i) % self.size
    def insert(self, key):
        i = 0
        while i < self.size:
            pos = self._hash(key, i)
            if self.table[pos] is None:
                self.table[pos] = key
                return
            i += 1
        raise Exception("Table full")
```

Double Hashing - $h(k, i) = (h_1(k) + i * h_2(k)) \bmod m$: Ավելի լավ է պատահականացնում զոնդավորման հաջորդականությունը, թույլ է տալիս ավելի քիչ կլաստերացում:

```
def double_hash_insert(table, key, h1, h2):
    m = len(table)
    i = 0
    while i < m:
        pos = (h1(key) + i * h2(key)) % m
        if table[pos] is None:
            table[pos] = key
            return
        i += 1
    raise Exception("Table full")
```

3. Dynamic Programming

Դիտարկենք ծրագրավորումը կիրառվում է օպտիմալացման խնդիրների համար, որոնք ունեն **օպտիմալ ենթակառուցվածք** (ենթախնդիրների օպտիմալ լուծումներից կազմվում է ընդհանուրի օպտիմալը) և **վերադարձող ենթախնդիրներ** (նույն ենթախնդիրները բազմիցս են հանդիպում): Գաղափարն է յուրաքանչյուր ենթախնդիր լուծել մեկ անգամ և պահել աղյուսակում (memoization կամ bottom-up):

3.1 Ֆիբոնաչիի թվեր (Fibonacci)

$F(0)=0$, $F(1)=1$, $F(n)=F(n-1)+F(n-2)$:

Պարզ ռեկուրսիան էքսպոնենցիալ է $O(2^n)$:

```
def fib_naive(n:int) -> int:
    if n <= 1:
        return n
    return fib_naive(n - 2) + fib_naive(n - 1)
```

Դիտարկենք ծրագրավորմամբ Ֆիբոնաչիի խնդիրը լուծվում է $O(n)$ ժամանակում:

```
# # Memoization (Top-Down) -> recursive + caching -> O(n)
def fib_memoization(n:int, memo:dict = None) -> int:
    # Time Complexity: O(n)
    # Space Complexity: O(n)
    if memo is None:
        memo = {}

    if n in memo:
        return memo[n]

    if n <= 1:
        return n

    memo[n] = fib_memoization(n - 1, memo) + fib_memoization(n - 2, memo)
    return memo[n]

# # Tabulation (Bottom-Up) -> O(n)
def fib_tabulation(n:int) -> int:
    # Time Complexity: O(n)
```

```

    # Space Complexity: O(n)
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]

def fib_optimized(n: int) -> int:
    # Time Complexity: O(n)
    # Space Complexity: O(1)
    if n <= 1:
        return n
    prev2, prev1 = 0, 1
    for _ in range(2, n + 1):
        curr = prev1 + prev2
        prev2, prev1 = prev1, curr
    return prev1

```

Fast-Doubling ալգորիթմը հիմնված է **Մաթեմատիկական ինքնուրույնների և բաժանման մեթոդի (Divide and Conquer)** վրա:

```

def fib_fast_doubling(n:int) -> int:
    # Time Complexity: O(log(n))
    # Space Complexity: O(log(n))
    if n == 0:
        return (0, 1)

    a, b = fib_fast_doubling(n >> 1) # a = F(k), b = F(k+1); where k = n
    // 2
    c = a * (2 * b - a) # F(2k)
    d = a * a + b * b # F(2k + 1)

    if n & 1: # n is odd
        return (d, c + d)
    else: # n is even
        return (c, d)

def fib(n:int) -> int:
    return fib_fast_doubling(n)[0]

```

3.2 Rod Cutting Problem

Տրված է n երկարությամբ ձող և գներ $p[i]$ ($i=1..n$): Պետք է կտրել ձողը կտորների այնպես, որ ընդհանուր եկամուտը լինի առավելագույնը: Օպտիմալ արժեքը՝ $r[n] = \max_{\{1 \leq i \leq n\}} (p[i] + r[n-i])$:

```
def rod_cutting_naive(length: int, prices: list) -> int:
    """
    Time Complexity:  $O(2^n)$ 
    Space Complexity:  $O(n)$  - recursion stack depth
    """
    if length <= 0:
        return 0

    max_val = float('-inf')
    for i in range(1, length + 1):
        if i <= len(prices):
            res = prices[i - 1] + rod_cutting_naive(length - i, prices)
            max_val = max(max_val, res)

    return max_val if max_val != float('-inf') else 0
```

Բացատրություն:

- Յուրաքանչյուր քայլում կատարում ենք length հնարավոր կտրվածքներ:
- Ռեկուրսիայի ծառն ունի 2^n հանգույց
- **Վատագույն դեպքում** աշխատում է էքսպոնենցիալ:

Memoization - Top-Down

```
def rod_cutting_memoization(length: int, prices: list) -> int:
    """
    Time Complexity:  $O(n^2)$ 
    Space Complexity:  $O(n)$ 
    """
    memo = [-1] * (length + 1)
    memo[0] = 0

    def helper(l: int) -> int:
        if l <= 0:
            return 0
        if memo[l] != -1:
            return memo[l]
```

```

        return memo[l]

    max_val = 0
    for i in range(1, l + 1):
        if i <= len(prices):
            max_val = max(max_val, prices[i - 1] + helper(l - i))

    memo[l] = max_val
    return max_val

return helper(length)

```

Բացատրություն:

- Յուրաքանչյուր ենթախնդիր (0-ից մինչև length) հաշվարկվում է **միայն մեկ անգամ**
- Length հնարավոր ենթախնդիրներ, յուրաքանչյուրը $O(n)$ աշխատանք => $O(n^2)$
- Memo-ի շնորհիվ խուսափում ենք կրկնվող հաշվարկներից:

Tabulation - Bottom-Up

```

def rod_cutting_tabulation(length: int, prices: list) -> int:
    """
    Time Complexity:  $O(n^2)$ 
    Space Complexity:  $O(n)$ 
    """
    dp = [0] * (length + 1)

    for i in range(1, length + 1):
        max_val = 0
        for j in range(1, i + 1):
            if j <= len(prices):
                max_val = max(max_val, prices[j - 1] + dp[i - j])
        dp[i] = max_val

    return dp[length]

```

Բացատրություն:

- DP աղյուսակը լրացվում է 1-ից մինչև length
- Յուրաքանչյուր i-ի համար դիտարկում ենք բոլոր հնարավոր առաջին կտրվածքներ (1-ից i)
- Երկու ցիկլ - $O(n^2)$

3.3 Coin Change Problem

Տրված է մետաղադրամների տեսակներ (անվերջ քանակով) և գումար amount: Պետք է գտնել նվազագույն մետաղադրամների քանակը, որով կարելի է կազմել amount (կամ -1, եթե անհնար է):

$dp[a] = \min(dp[a-c] + 1)$ բոլոր $c \leq a$ -ի համար

```
def coin_change_naive(coins: list, amount: int) -> int:
    """
    Time Complexity:  $O(m^n)$  where  $m = \text{len}(\text{coins})$ ,  $n = \text{amount}$ 
    Space Complexity:  $O(n)$  - recursion depth
    """
    if amount == 0:
        return 0
    if amount < 0:
        return float('inf')

    min_coins = float('inf')
    for coin in coins:
        res = coin_change_naive(coins, amount - coin)
        if res != float('inf'):
            min_coins = min(min_coins, res + 1)

    return min_coins if min_coins != float('inf') else -1
```

Բացատրություն:

- Յուրաքանչյուր քայլում amount-ը նվազում է
- Ռեկուրսիայի ծառը շատ մեծ է (m ճյուղ յուրաքանչյուր հանգույցում)

Memoization - Top-Down

```
def coin_change_memoization(coins: list, amount: int) -> int:
    """
    Time Complexity:  $O(m \times n)$  where  $m = \text{len}(\text{coins})$ ,  $n = \text{amount}$ 
    Space Complexity:  $O(n)$ 
    """
    memo = [-1] * (amount + 1)
    memo[0] = 0
```



```
def helper(a: int) -> int:
    if a < 0:
        return float('inf')
    if memo[a] != -1:
        return memo[a]

    min_coins = float('inf')
    for coin in coins:
        res = helper(a - coin)
        if res != float('inf'):
            min_coins = min(min_coins, res + 1)

    memo[a] = min_coins
    return min_coins

result = helper(amount)
return result if result != float('inf') else -1
```

Բացատրություն.

- amount+1 հնարավոր ենթախնդիրներ (0-ից մինչև amount)
- Յուրաքանչյուր ենթախնդրի համար դիտարկում ենք m հնարավոր մետաղադրամ
- Ընդհանուր $O(m \times n)$
- Memo-ի շնորհիվ յուրաքանչյուր ենթախնդիր հաշվվում է միայն մեկ անգամ

Tabulation - Bottom-Up

```
def coin_change_tabulation(coins: list, amount: int) -> int:
    """
    Time Complexity:  $O(m \times n)$ 
    Space Complexity:  $O(n)$ 
    """
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0

    for i in range(1, amount + 1):
        for coin in coins:
            if i - coin >= 0:
                dp[i] = min(dp[i], dp[i - coin] + 1)

    return dp[amount] if dp[amount] != float('inf') else -1
```

Բացատրություն.

- DP աղյուսակը լրացվում է 1-ից մինչև amount
- Յուրաքանչյուր i-ի համար փորձում ենք բոլոր m մետաղադրամները
- Երկու gիկլ $\rightarrow O(m \times n)$
- Bottom-up, ռեկուրսիա չկա

3.4 Knapsack Problem

Տրված են n իրեր, յուրաքանչյուրն ունի քաշ w_i և արժեք v_i , և պայուսակի տարողություն W: Պետք է ընտրել իրը (0 կամ 1 անգամ) այնպես, որ ընդհանուր քաշը $\leq W$, իսկ ընդհանուր արժեքը՝ առավելագույնը: $dp[i][w] = \max(dp[i-1][w], dp[i-1][w - w_i] + v_i)$

```
def knapsack_naive(weights: list, values: list, capacity: int, n: int = None) -> int:
    """
    Time Complexity:  $O(2^n)$  - յուրաքանչյուր իրը կամ վերցնում ենք, կամ չենք
    վերցնում
    Space Complexity:  $O(n)$  - recursion stack depth
    """
    if n is None:
        n = len(weights)

    if n == 0 or capacity == 0:
        return 0

    if weights[n-1] > capacity:
        return knapsack_naive(weights, values, capacity, n-1)

    return max(
        values[n-1] + knapsack_naive(weights, values, capacity - weights[n-1], n-1),
        knapsack_naive(weights, values, capacity, n-1)
    )
```

Բացատրություն.

- Յուրաքանչյուր իրի համար կա 2 տարբերակ (վերցնել/չվերցնել) $\rightarrow 2^n$ հնարավոր կոմբինացիա
- Ռեկուրսիայի ծառը ունի 2^n հանգույց
- $n=30$ -ի դեպքում արդեն ~ 1 միլիարդ կանչ $\rightarrow$ գործնականում անօգտագործելի

```

def knapsack_memoization(weights: list, values: list, capacity: int) ->
int:
    """
    Time Complexity:  $O(n \times W)$ , where  $n$  = number of items,  $W$  = capacity
    Space Complexity:  $O(n \times W) + O(n)$  recursion stack
    """
    n = len(weights)
    memo = [[-1 for _ in range(capacity + 1)] for _ in range(n + 1)]

    def helper(idx: int, remaining_cap: int) -> int:
        if idx == 0 or remaining_cap == 0:
            return 0
        if memo[idx][remaining_cap] != -1:
            return memo[idx][remaining_cap]

        if weights[idx-1] > remaining_cap:
            memo[idx][remaining_cap] = helper(idx-1, remaining_cap)
        else:
            memo[idx][remaining_cap] = max(
                values[idx-1] + helper(idx-1, remaining_cap -
weights[idx-1]),
                helper(idx-1, remaining_cap)
            )
        return memo[idx][remaining_cap]

    return helper(n, capacity)

```

Բացատրություն.

- Կան $(n+1) \times (W+1)$ հնարավոր ենթախնդիրներ
- Յուրաքանչյուր ենթախնդիր հաշվվում է միայն մեկ անգամ
- $O(n \times W)$ – սա արդեն բավականին արագ է (օրինակ $n=100$, $W=1000 \rightarrow 100,000$ գործողություն)

```

def knapsack_tabulation(weights: list, values: list, capacity: int) ->
int:
    """
    Time Complexity:  $O(n \times W)$ 
    Space Complexity:  $O(n \times W)$ 
    """
    n = len(weights)

```

```

dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]

for i in range(1, n + 1):
    for w in range(1, capacity + 1):
        if weights[i-1] <= w:
            dp[i][w] = max(
                values[i-1] + dp[i-1][w - weights[i-1]],
                dp[i-1][w]
            )
        else:
            dp[i][w] = dp[i-1][w]

return dp[n][capacity]

```

Բացատրություն.

- DP աղյուսակը լրացվում է տող առ տող (item առ item)
- Յուրաքանչյուր i -ի և w -ի համար $O(1)$ աշխատանք
- Ընդհանուր $O(n \times W)$

4. Greedy Algorithms

Greedy Algorithm-ը յուրաքանչյուր քայլում կատարում է **լոկալ օպտիմալ** ընտրություն՝ հույս ունենալով, որ այն կհանգեցնի գլոբալ օպտիմալի: Այն աշխատում է միայն այն խնդիրների համար, որոնք ունեն **ազահ ընտրության հատկություն** և **օպտիմալ ենթակառուցվածք**:

4.1 Activity Selection

Տրված են n գործողություններ, սկիզբ՝ s_i և ավարտ՝ f_i (f_i -երը դասավորված են աճման կարգով): Պետք է ընտրել առավելագույն քանակությամբ չհամընկնող գործողություններ: Ազահ ընտրությունն է՝ միշտ վերցնել ամենավաղ ավարտվող գործողությունը, որը համատեղելի է արդեն ընտրվածների հետ: Աշխատում է $O(n)$ ժամանակում (նախապես դասավորելուց հետո):

```

def activity_selection(start, finish):
    # assume that finish is already sorted

    n = len(start)
    selected = [0] # index of the first activity
    last_finish = finish[0]

```

```

for i in range(1, n):
    if start[i] >= last_finish:
        selected.append(i)
        last_finish = finish[i]

return selected

```

4.2 Huffman Coding

Սեղմման ալգորիթմ, որը կառուցում է օպտիմալ նախաճանցային կոդ՝ հիմնվելով սիմվոլների հաճախականությունների վրա: Ազահ քայլն է՝ ամենաքիչ հաճախականությամբ երկու հանգույցներ միավորել մեկում (կրկնել մինչև մնա մեկ հանգույց): Ժամանակը $O(n \log(n))$ (օգտագործելով min-heap):

```

import heapq

# # Huffman Coding

class Node:
    def __init__(self, char, freq):
        self.char = char
        self.freq = freq
        self.left = None
        self.right = None

    def __lt__(self, other):
        return self.freq < other.freq

def build_huffman_tree(frequencies):
    # # Priority queue for each character
    heap = [Node(char, freq) for char, freq in frequencies.items()]
    heapq.heapify(heap)

    while len(heap) > 1:
        node1 = heapq.heappop(heap)
        node2 = heapq.heappop(heap)

        merged = Node(None, node1.freq + node2.freq)
        merged.left = node1
        merged.right = node2

```

```

        heapq.heappush(heap, merged)

    return heap[0] # root

def huffman_encode(node, current_code = "", codes = {}):
    if node is None:
        return

    if node.char is not None:
        codes[node.char] = current_code

    huffman_encode(node.left, current_code + "0", codes)
    huffman_encode(node.right, current_code + "1", codes)

    return codes

def huffman_decode(root, encoded_str):
    decoded_output = ""
    current_node = root

    for bit in encoded_str:
        if bit == "0":
            current_node = current_node.left
        else:
            current_node = current_node.right

        if current_node.left is None and current_node.right is None:
            decoded_output += current_node.char
            current_node = root

    return decoded_output

```

4.3 Minimum Spanning Tree

Prim Algorithm - անեցնում է մեկ ծառ, յուրաքանչյուր քայլում ավելացնելով ամենակարճ եզրը, որը միացնում է ծառը մնացած գագաթներին: Իրականացվում է min-heap-ով, ժամանակը՝ $O(E \log V)$:

```

import heapq

def prim(graph, start):
    visited = set()
    heap = [(0, start, None)]  # (weight, vertex, parent)
    mst = []
    while heap:
        w, u, parent = heapq.heappop(heap)
        if u in visited:
            continue
        visited.add(u)
        if parent is not None:
            mst.append((parent, u, w))
        for v, weight in graph[u]:
            if v not in visited:
                heapq.heappush(heap, (weight, v, u))
    return mst

```

Kruskal Algorithm - դասավորում է բոլոր եզրերը ըստ կշռի աճման կարգով, ապա ավելացնում է եզր, եթե այն միացնում է տարբեր կոմպոնենտներ (օգտագործելով disjoint-set union): ժամանակը՝ $O(E \log E)$:

```

def kruskal(edges, n):
    edges.sort(key = lambda e : e[2])
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    def union(x, y):
        rx, ry = find(x), find(y)

        if rx == ry:
            return False
        parent[ry] = rx
        return True

```

```
mst = []  
for u, v, w in edges:  
    if union(u, v):  
        mst.append((u,v,w))  
return mst
```